

Práctica Kubernetes con Minikube

Deployments, Services, Ingress y rolling updates en ARM64

Dato	Valor
Estudiante	Natalia Hernández
Fecha	9 de septiembre de 2026
Plataforma	Ubuntu 22.04 · ARM64 · Docker
Versiones	Minikube 1.39.0 · Kubernetes/kubectl 1.37.0

PRÁCTICA COMPLETA · INGRESS Y ROLLING UPDATE VERIFICADOS

Arquitectura

Capa	Elemento	Función
Host	servidorUbuntu	VM ARM64 192.168.100.3
Clúster	Minikube	Control plane de un nodo
Entrada	Ingress NGINX	Rutas /node y /echo
Aplicaciones	hello-node / echo-server	Deployments y Services

Resultados verificados

Prueba	Resultado	Detalle
Nodo	Ready	Kubernetes 1.37.0 ARM64
hello-minikube	HTTP correcto	NodePort 30081
Dashboard	HTTP 200	Proxy temporal 8001
Port-forward	Respuesta v1	Puerto 8090
Escalado	4 pods	4 IP diferentes
Balanceo	4 pods observados	12 peticiones
Ingress	2 rutas correctas	/node y /echo
Rolling update	v1 → v2	4 réplicas disponibles
Limpieza	Minikube detenido	Contenedores restaurados

1. Objetivo

Comprender Kubernetes mediante un clúster local de un nodo, despliegues, servicios, observación, escalado, balanceo, Ingress y actualizaciones progresivas.

2. Entorno y adaptaciones ARM64

La práctica se ejecutó en servidorUbuntu, una máquina Ubuntu 22.04 ARM64 con dos CPU y cerca de 4 GB de memoria. Docker 29.8.0 ya estaba instalado.

Se instalaron Minikube 1.39.0 y kubectl 1.37.0 para ARM64. Minikube inició Kubernetes 1.37.0 con containerd 2.3.4 y el controlador Docker. La aplicación Node usa node:22-alpine en lugar de Node 4.4, una versión obsoleta.

3. Clúster y hello-minikube

El nodo minikube alcanzó estado Ready con IP 192.168.49.2. Se registraron la información del clúster, los pods del sistema, eventos y la configuración activa de kubectl.

hello-minikube se desplegó con kicbase/echo-server:1.0 y un Service NodePort en el puerto 30081. La respuesta identificó el pod que atendió la solicitud y se verificaron sus registros.

4. Dashboard y acceso

Se habilitó el complemento Kubernetes Dashboard. Un proxy temporal en 192.168.100.3:8001 respondió HTTP 200. El proxy se cerró al terminar.

hello-node se expuso con NodePort 30080. También se ejecutó kubectl port-forward sobre el Service en 192.168.100.3:8090 y la aplicación devolvió correctamente el nombre del pod y la versión v1.

5. Aplicación hello-node

La imagen se construyó dentro del runtime de Minikube. Kubernetes desplegó hello-node:v1 y se comprobaron pods, Deployment, Service, descripción, registros, variables de entorno, código fuente y una petición ejecutada dentro del contenedor.

6. Escalado y balanceo

El Deployment pasó de una a cuatro réplicas, cada una con una IP diferente. Doce peticiones sucesivas fueron atendidas por los cuatro nombres de pod, lo que demostró el balanceo del Service.

Después se redujo el Deployment a dos réplicas. Kubernetes mantuvo dos pods Running y terminó los otros dos.

7. Ingress Controller

Un Ingress Controller observa recursos Ingress y configura un proxy de entrada para dirigir tráfico HTTP o HTTPS hacia Services internos.

Se habilitó Ingress NGINX y se creó el host practica.local. La ruta /node respondió desde hello-node y /echo respondió desde hello-minikube usando la misma IP de entrada.

8. Rolling update

Como desafío se construyeron hello-node:v1 y hello-node:v2. Con cuatro réplicas activas, kubectl set image inició una actualización progresiva. Kubernetes sustituyó los pods por etapas y mantuvo instancias disponibles.

El historial registró dos revisiones. Ocho peticiones posteriores respondieron con v2 desde diferentes pods, confirmando la actualización y el balanceo.

9. Material de GPU

El ZIP adicional usa NVIDIA CUDA, NumPy y Numba. Una prueba real requiere una GPU NVIDIA visible, sus controladores Linux y NVIDIA Container Toolkit. VirtualBox sobre este Mac Apple Silicon no expone ese dispositivo, por lo que los archivos se conservaron como referencia y no se alteró Docker con una instalación que no podría usar aceleración.

10. Limpieza y restauración

Se eliminaron los Deployments, Services e Ingress creados. Solo quedó el Service interno kubernetes. El proxy del Dashboard se cerró y Minikube quedó completamente detenido, conservando su perfil.

Los cuatro contenedores que estaban activos antes de la práctica se reiniciaron. Flask + MySQL y Flask + Redis respondieron con estado ok, por lo que el trabajo anterior quedó restaurado.

11. Conclusión

La práctica demostró el ciclo completo de una aplicación Kubernetes: construcción de imagen, despliegue, exposición, inspección, escalado, balanceo, entrada HTTP, actualización gradual y limpieza segura.

Referencias oficiales

- <https://minikube.sigs.k8s.io/docs/start/>
- <https://minikube.sigs.k8s.io/docs/drivers/docker/>
- <https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/>
- <https://kubernetes.io/docs/tasks/access-application-cluster/ingress-minikube/>
- <https://kubernetes.io/docs/tutorials/kubernetes-basics/update/update-intro/>
- <https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html>